



International Islamic University Chittagong

Department of Computer Science and Engineering

Lab Report

Lab No 5: Implementation of the Trigonometric Circle Algorithm

Course Title: Computer Graphics Lab

Course Code: CSE-4742

Session: Autumn 2026

Semester: 7th

Section: 7BF

Student Name	Tahasina Tasnim Afra
Student ID	C233456
Department	Computer Science and Engineering

Submitted To

Ms. Sultana Tasnim Jahan
Lecturer, Department of CSE, IIUC

Submission Date:

September 20, 2026

1. Number & Name of the Experiment

Lab 5: Implementation of Trigonometric Circle Algorithm

2. Introduction

The Trigonometric Circle Algorithm is a scan-conversion method that uses polar coordinates and trigonometric functions to generate the pixels of a circle. Instead of calculating the circle using its Cartesian equation, the algorithm determines the coordinates of each point using the angle θ .

For a circle with center (h, k) and radius r , the coordinates of a point on the circle are given by

$$x = h + r \cos \theta$$

and

$$y = k + r \sin \theta.$$

The algorithm starts from $\theta = 0$ and increases the angle by a fixed step size until $\theta = \pi/4$, which corresponds to 45° . Only one octant of the circle is calculated directly, and eight-way symmetry is then used to generate the complete circle.

Since the method uses sine and cosine functions and floating-point calculations, it directly follows the polar-coordinate representation of a circle.

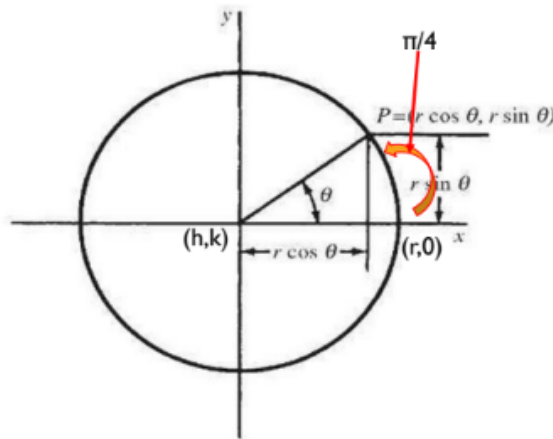


Figure 1: Trigonometric method for calculating a circle point

3. Algorithm

The Trigonometric Circle Algorithm generates points on a circle using the polar-coordinate equations

$$x = r \cos \theta$$

and

$$y = r \sin \theta.$$

For a circle centered at (h, k) , the center coordinates are added to the calculated values. The algorithm calculates one octant from $\theta = 0$ to $\theta = \pi/4$ and uses eight-way symmetry to obtain the remaining points.

Initial Values

The algorithm initializes the radius, center, angular step size, and starting angle. The starting angle is

$$\theta = 0.$$

The ending angle is

$$\theta_{end} = \frac{\pi}{4}.$$

For the selected circle, the radius is

$$r = 200$$

and the screen center is

$$(h, k) = (400, 300).$$

Calculation of Coordinates

For every value of θ , the coordinates are calculated using

$$x = r \cos \theta$$

and

$$y = r \sin \theta.$$

For a circle with center (h, k) , the corresponding screen coordinates are

$$x = h + r \cos \theta$$

and

$$y = k + r \sin \theta.$$

The calculated values are rounded to obtain integer pixel coordinates before they are plotted.

Angular Step

After calculating and plotting a point, the angle is increased by a fixed step size:

$$\theta = \theta + \Delta\theta.$$

In the implementation, a small angular step is used so that points are generated at regular intervals along the circle.

Eight-Way Symmetry

After calculating a point (x, y) , the remaining seven points are obtained using eight-way symmetry. This is possible because a circle has symmetry about the horizontal, vertical, and diagonal axes.

For a circle centered at (h, k) , the eight symmetric points are

$$\begin{array}{ll} (h + x, k + y), & (h + y, k + x), \\ (h - y, k + x), & (h - x, k + y), \\ (h - x, k - y), & (h - y, k - x), \\ (h + y, k - x), & (h + x, k - y). \end{array}$$

Stopping Condition

The direct calculation continues until

$$\theta > \frac{\pi}{4}.$$

Since

$$\frac{\pi}{4} = 45^\circ,$$

the algorithm calculates only one octant of the circle. At this angle,

$$\cos \frac{\pi}{4} = \sin \frac{\pi}{4} = \frac{1}{\sqrt{2}}.$$

Therefore,

$$x \approx y \approx \frac{r}{\sqrt{2}}.$$

For $r = 200$,

$$x \approx y \approx 141.42.$$

Implementation

The implementation follows the polar-coordinate method. The `TrigonometricCircle()` function initializes the radius, center, starting angle, angular step size, and ending angle. It then incrementally increases θ and calculates the corresponding coordinates using $x = r \cos \theta$ and $y = r \sin \theta$.

The calculated coordinates are rounded to obtain integer pixel positions and passed to `plotCirclePoints()`, which generates the eight symmetric points using `putpixel()`. The program also displays the calculated angle and (x, y) coordinates using `cout`.

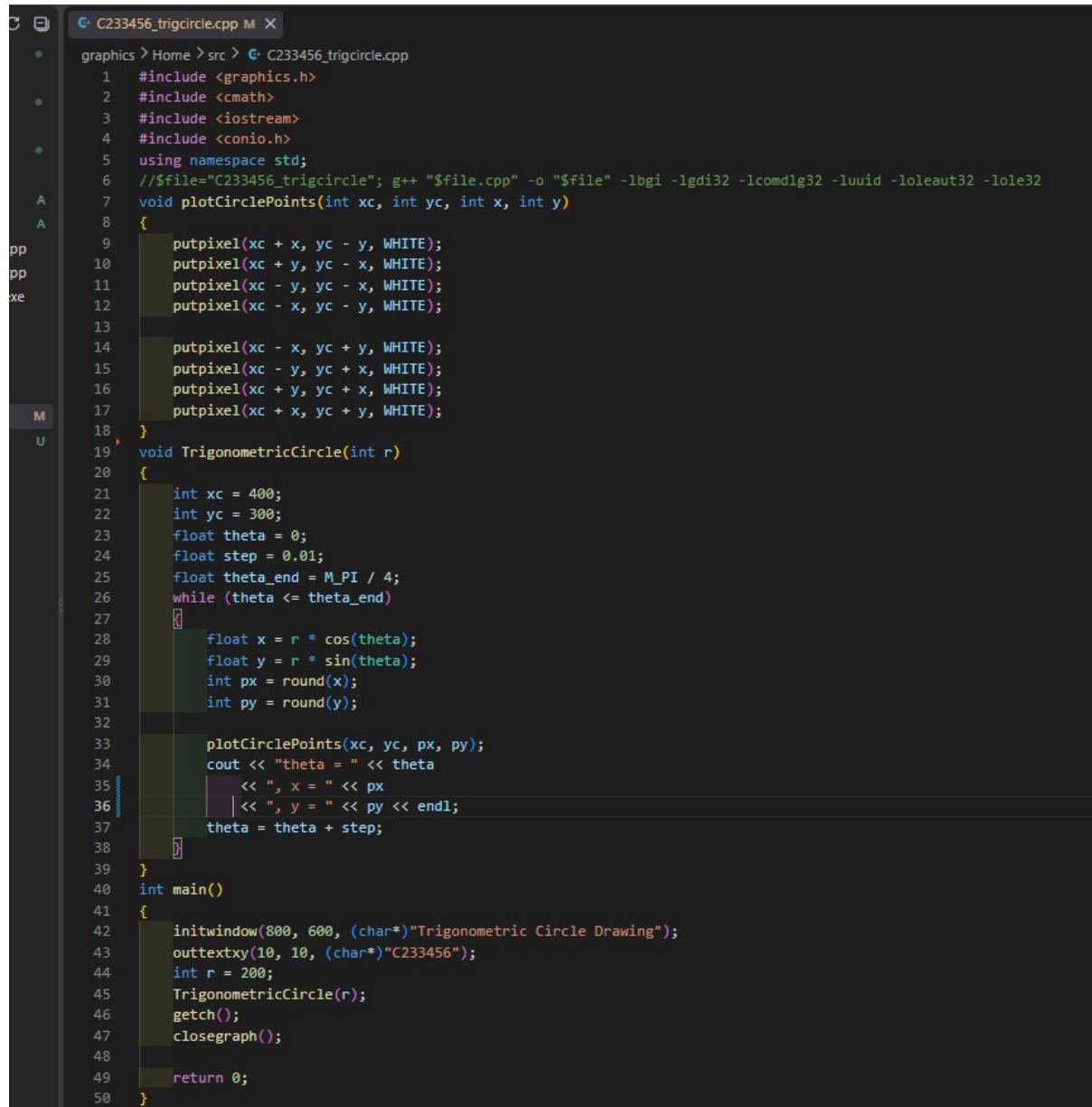
4. Program with Screenshot

Code

The C++ implementation of the Trigonometric Circle Algorithm is available in the `AFRA-comp-graphics` GitHub repository. The code used for the experiment is shown below.

Code Screenshot

The following screenshot shows the C++ implementation of the Trigonometric Circle Algorithm:

The screenshot shows a C++ code editor with a dark theme. The file is named 'C233456_trigcircle.cpp'. The code implements the Trigonometric Circle Algorithm. It includes headers for graphics, cmath, iostream, and conio. It uses the std namespace. A compile command is shown as a comment. The 'plotCirclePoints' function takes center coordinates (xc, yc) and point coordinates (x, y) and plots eight points around the center using 'putpixel'. The 'TrigonometricCircle' function takes a radius 'r', sets center coordinates to (400, 300), and iterates from theta = 0 to M_PI/4, calculating points (px, py) and plotting them. The 'main' function sets up a graphics window, calls 'TrigonometricCircle', and waits for a key press before closing the window.

```
1 #include <graphics.h>
2 #include <cmath>
3 #include <iostream>
4 #include <conio.h>
5 using namespace std;
6 // $file="C233456_trigcircle.cpp"; g++ "$file.cpp" -o "$file" -lbg -lstdc++ -lcomdlg32 -luuid -loleaut32 -lole32
7 void plotCirclePoints(int xc, int yc, int x, int y)
8 {
9     putpixel(xc + x, yc - y, WHITE);
10    putpixel(xc + y, yc - x, WHITE);
11    putpixel(xc - y, yc - x, WHITE);
12    putpixel(xc - x, yc - y, WHITE);
13
14    putpixel(xc - x, yc + y, WHITE);
15    putpixel(xc - y, yc + x, WHITE);
16    putpixel(xc + y, yc + x, WHITE);
17    putpixel(xc + x, yc + y, WHITE);
18 }
19 void TrigonometricCircle(int r)
20 {
21     int xc = 400;
22     int yc = 300;
23     float theta = 0;
24     float step = 0.01;
25     float theta_end = M_PI / 4;
26     while (theta <= theta_end)
27     {
28         float x = r * cos(theta);
29         float y = r * sin(theta);
30         int px = round(x);
31         int py = round(y);
32
33         plotCirclePoints(xc, yc, px, py);
34         cout << "theta = " << theta
35              << ", x = " << px
36              << ", y = " << py << endl;
37         theta = theta + step;
38     }
39 }
40 int main()
41 {
42     initwindow(800, 600, (char*)"Trigonometric Circle Drawing");
43     outtextxy(10, 10, (char*)"C233456");
44     int r = 200;
45     TrigonometricCircle(r);
46     getch();
47     closegraph();
48
49     return 0;
50 }
```

Figure 2: C++ implementation of the Trigonometric Circle Algorithm

5. Output/Result

The program was successfully executed and generated a complete circle using the Trigonometric Circle Algorithm. The coordinates of the circle points were calculated using the polar-coordinate equations $x = r \cos \theta$ and $y = r \sin \theta$. The calculated points were rounded and plotted using `putpixel()`.

For the selected radius of 200, the calculation started at $\theta = 0$, corresponding to the point (200,0) relative to the circle center, and continued up to approximately $\theta = \pi/4$. The remaining points were generated using eight-way symmetry. The student ID (C233456) was also displayed in the graphics window, and the complete circle was successfully generated.

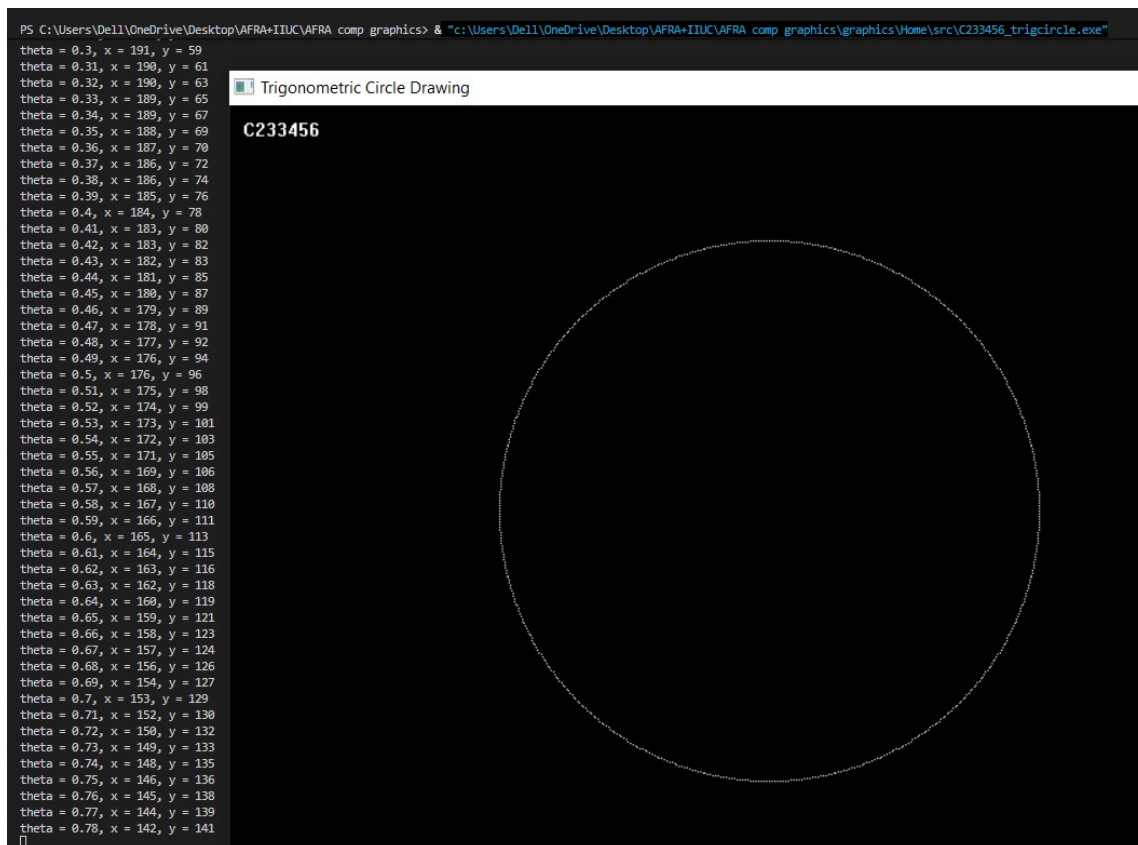


Figure 3: Output of the Trigonometric Circle Algorithm for radius $r = 200$

6. Discussion

The Trigonometric Circle Algorithm successfully generated a complete circle by calculating points using the polar-coordinate equations $x = r \cos \theta$ and $y = r \sin \theta$. The angle was increased incrementally from 0 to $\pi/4$, and the calculated points were converted into integer pixel coordinates.

Only one octant of the circle was calculated directly, while eight-way symmetry was used to generate the remaining points. This reduces the amount of direct calculation required to generate the complete circle.

The method is straightforward and follows the mathematical representation of a circle using polar coordinates. However, it requires repeated sine and cosine calculations as well as floating-point operations, which makes it less computationally efficient than incremental integer-based algorithms such as Bresenham's Circle Algorithm.

The time required depends on the selected angular step size. A smaller step produces more calculated points and requires more iterations, while a larger step reduces the number of calculations but may produce a less smooth result. The algorithm uses $O(1)$ additional space apart from the graphics output.

7. Conclusion

The Trigonometric Circle Algorithm was successfully implemented in C++ using the `graphics.h` library. The experiment demonstrated how polar coordinates and trigonometric functions can be used to calculate points on a circle using the equations $x = r \cos \theta$ and $y = r \sin \theta$.

The algorithm calculated one octant of the circle by incrementing θ from 0 to $\pi/4$. The calculated coordinates were rounded to obtain suitable pixel positions, and eight-way symmetry was then used to generate the remaining points of the complete circle.

The experiment also demonstrated how the mathematical coordinates can be mapped to the graphics window and displayed as individual pixels using `putpixel()`. Overall, the implementation provided a clear understanding of polar-coordinate-based circle scan conversion, angular point generation, and symmetry-based pixel plotting.